Projektuppgift

Introduktion till webbutveckling med HTML, CSS och JavaScript, DT224G - HT2026

Linneah Olofsson

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Linneah Olofsson, liol2603@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

1 Sammanfattning

Blåklockans Ridklubb är en fiktiv ridskola vars webbplats främst syftar till att underlätta den dagliga hanteringen av hästarna. De flesta funktioner på webbplatsen hämtar data från en separat JSON-fil där information om varje häst finns lagrad. Genom en kombination av HTML, CSS och Javascript skapas en strukturerad och interaktiv plattform.

Webbplatsens kärnfunktioner inkluderar ett filtreringsbart hästgalleri, en hagkarta samt ett kontaktformulär för medlemsansökningar. Projektet inleddes med en noggrann planeringsfas där två wireframes skapades samt en mockup togs fram i verktyget Figma. Fokus låg på att skapa en responsiv webbplats med en media query som anpassade sidans layout beroende på om sidan visades på mobil eller desktop.

Under projektets gång har utvecklingen mött problem som visat sig bli avgränsningar från den ursprungliga planen. Bland dessa problem fanns hagkartan som egentligen var tänkt att visa information genom att hovra med musen över en hage, men pga. Bristande kunskap inom svg-filer fick lösningen bli en klickbar karta.

2 Innehållsförteckning

1 Sammanfattning ii
2 Innehållsförteckning iii
3 Introduktion 1
3.1 Bakgrund och problemmotivering 1
3.2 Detaljerad problemformulering 1
3.3 Avgränsningar 1
4 Bakgrund 3
5 Metod 5
5.1 Tester 6
6 Konstruktion 7
7 Resultat 11
7.1 Testresultat 11
8 Slutsatser 12

3 Introduktion

3.1 Bakgrund och problemmotivering

Hästnäringen i Sverige innefattar idag cirka 450 ridskolor belägna över hela landet [1]. Många ridskolor har system där de ridande medlemmarna förväntas hjälpa till med hästarnas skötsel under helgerna enligt ett rullande schema. I många fall kan detta innebära osäkerhet för den enskilda medlemmen kring aktuella rutiner.

Webbplatsen till den fiktiva ridskolan Blåklockans ridklubb är utvecklad för att medlemmar enkelt ska kunna hämta information om både enskilda hästar och allmänna stallrutiner. Fördelen med ett digitalt system för detta är att de medlemmar som känner sig osäkra har möjlighet att se över gällande information i förväg. I förlängningen kan det även vara en viktig funktion för ridskolans personal som ges möjlighet att uppdatera informationen löpande utan att behöva vara på plats i stallet.

3.2 Detaljerad problemformulering

Jag har valt att inkludera några funktioner som jag anser viktiga och tidsbesparande i jämförelse med en situation där information finns skriftligt i olika delar av stallet. De funktioner jag valt att inkludera är följande:

- Ett galleri som visar klickbara bilder på alla hästar med möjlighet att filtrera dessa efter kön eller storlek.
- En dialogruta med all information om respektive häst vid klick på dess bild i galleriet.
- En interaktiv karta över hagar som visar vilka hästar som ska gå i vilken hage samt hur mycket hö dessa hästar äter tillsammans.
- Väderdata som uppdateras dagligen. Utifrån denna data visas en lista med vilka hästar som behöver ha täcken på sig.
- En att göra-lista som ett verktyg för att själv kunna bocka av de uppgifter man utfört.
- Ett kontaktformulär för att kunna anmäla sitt intresse för att börja rida på ridskolan.

3.3 Avgränsningar

På grund av begränsade kunskaper inom skapande av SVG-bilder kunde inte den interaktiva kartan visa informationen genom hover som det först var tänkt. Lösningen blev en klickbar karta.

Även begränsningen att localstorage endast sparas i ens egen webbläsare gjorde att vissa tilltänkta funktioner inte gick att tillämpa i praktiken. Dessa hade kunnat lösas med databaser, men där brister mina kunskaper.

4 Bakgrund

API – ”Application Programming Interface” tillåter programmerare att hämta data och funktioner från olika webbtjänster och applikationer. [2]

Wireframe och mockup – Dessa begrepp ingår båda två i den inledande planeringsfasen av en webbplats. En wireframe är en grov skiss över layout och var tex bilder och text ska placeras. En mockup visar mer exakt vilka bilder, vilket typsnitt och vilka färger som ska användas etc. [3]

HTML – ”Hyper text markup language” som används för att strukturera innehåll på webben. HTML definerar vad som är vad på webbplatsen, exempelvis vad som är rubriker, vad som är listor och vad som är bilder. [3]

CSS – “Cascading style sheets” ansvarar för sidans layout och design. Med css kan man bestämma sidans färger, typsnitt, layout och mycket mer. [3]

Javascript – Används för att skapa dynamik och interaktivitet på webbsidor. Med Javascript kan man skapa funktioner som förändrar innehållet. Med detta kan man tex. hantera och validera formulär. [3]

Flexbox och CSS-grid – Används för att utforma en webbsidas layout. Flexbox tillåter utvecklaren att placera flera olika element (exempelvis flera bilder) på en rad **eller** i en kolumn. CSS-grid har nästan samma funktion men tillåter placering i rader **och** kolumner. [6]

Index – Förstasidan på en webbplats brukar döpas till index. [3] Alla senare omnämningar av index syftar till webbplatsens startsida.

JSON – ”Java script object notation” är ett lättviktigt format för att transportera och spara data. Javascript kan konvertera innehållet i JSON-filen och sedan använda innehållet i funktioner på webbplatsen. [13]

W3C – Den internationella organisation som tillhandahåller tekniska standarder, riktlinjer och protokoll för webbplatser. De har även verktyg för validering av html och css som används i projektet.

DOM – ”Document object model” representerar den hierarkiska strukturen av element i ett dokument och tillåter manipulering av dessa via tex. javascript. [3]

Media query – Ger oss möjlighet att ändra css baserat på egenskaper som tex. webbläsarens fönsterbredd. Det är vanligt att använda media queries för att göra webbplatser mer responsiva för användaren. [3]

5 Metod

Projektet inleddes genom att jag skapade två wireframes i anteckningsprogrammet notes + som har stöd för tecknade noteringar. Jag valde att göra mina wireframes i tecknad form för att kunna tillämpa snabba ändringar och enkelt kunna skissa upp mina idéer. Här försökte jag tydliggöra sidans struktur och tänka på grid, flexbox samt vilka semantiska element som kunde appliceras på varje del.

Utifrån mina wireframes övergick jag sedan till att skapa webbplatsens design genom en mockup i Figma. Jag valde att göra min mockup i anpassad storlek för desktop för att lättare kunna visualisera slutprodukten med tillämpad media query för stor skärm. Visionen var att skapa en webbplats vars färgprofil kunde knytas till ridklubbens namn Blåklockan utan att upplevas överdriven eller störande för ögonen. Målet var att skapa ett relativt stilrent utseende men samtidigt begränsa användningen av vitt. Sidans huvudsakliga färger blev därav olika nyanser av lila och en beige färg för sektioner.

Webbplatsens sektioner har den beigea färgen som bakgrund för att ordna innehållet och bidra till att ge användaren ett intryck av struktur. Eftersom just struktur är ett ledord för webbplatsens funktion ansåg jag att detta var passande för ändamålet.

På webbplatsens startsida finns en sektion dedikerad till FAQ för ridklubben. Trots att webbplatsens främsta målgrupp är de som redan är medlemmar så bedömer jag det som högst sannolikt att även potentiellt blivande medlemmar söker sig till klubbens hemsida om intresse uppstår. Att ha lättillgängliga svar på de vanligaste frågorna direkt på index blev därav mitt val för att försöka fånga dessa besökare direkt vid första besök på webbplatsen. För att hålla startsidan strukturerad även för de besökare som inte har intresse för dessa frågor ville jag att dessa skulle vara möjliga att visa och dölja. På index beslutade jag även att inkludera en sektion för nyheter då jag såg det som en möjlighet till att inkludera fler semantiska element i projektet.

Den första sidan visar enbart klickbara bilder på alla klubbens hästar. Varje klick genererar en informationsruta som hämtar data om hästen från en separat JSON-fil. Jag valde att endast ha bilderna med respektive hästnamn samt en ruta med filtrering för att hålla utseendet så simpelt som möjligt. I min mening skapade de olika bilderna nog stort intresse för ögat för att användarens uppmärksamhet ska fångas vid besök.

Gällande uppmärksamhet och bilder har jag valt att lägga vikt vid att webbplatsens bilder ska hålla låg filstorlek utan att tappa för mycket i kvalitet. Eftersom hästsidan innehåller många bilder ser jag det som av största vikt att de laddar in snabbt eftersom studier visar att 53% av besökare på en webbplats väljer att navigera vidare om sidan inte laddats fullt redan efter tre sekunder. [4] För detta har jag använt mig av bildformatet WebP som komprimerar bilder med mindre förlust av kvalitet i förhållande till PNG och JPEG. [5]

Undersidan stallinfo innehåller större delen av de funktioner jag tagit upp i avsnitt 3.2. Här föll beslutet på en horisontell layout med en interaktiv karta över stallets hagar i centrum. Eftersom kartan var den enda tänkta bilden på undersidan så kändes det logiskt att låta denna ta störst plats. Grundtanken var att skapa en liggande karta och låta den ta upp översta delen av hela sidan men vid konstruktion visade det sig snabbt att det skulle ta för mycket fokus från andra funktioner. I stället valde jag då att göra en stående karta och placera denna i centrum med övriga funktioner på varsin sida om denna.

Eftersom inhämtning av väderdata var nödvändig för funktionen som bedömer hästarnas täcken utnyttjades Open-meteos öppna API. Syftet med att hämta denna data var att kunna visa dagens medeltemperatur för användaren samt en ikon som indikerade vilken typ av väder som väntades för stunden. Med hjälp av detta skulle sedan en komplett lista skrivas ut på samtliga hästar som behöver täcke för dagen. Denna funktion gör att användaren slipper kontrollera täckesbehov för varje häst separat och blir därmed en tidsbesparande funktion.

Undersidans tredje och sista funktion är en lista med olika uppgifter som ska utföras i stallet under en dag. Tanken med denna var att medlemmen skulle få möjlighet att bocka av utförda uppgifter för att själv hålla reda på vad som blivit gjort och inte. Resultatet från denna var alltså aldrig tänkt att skickas eller lagras någon annanstans än för medlemmen själv.

För att återkoppla till behovet av att inkludera funktioner för att intressera blivande medlemmar så föll det sig naturligt att inkludera en funktion där dessa personer enkelt kan anmäla sitt intresse för att bli medlem. Detta görs via ett kontaktformulär på webbplatsens sista undersida. Här var målsättningen att skapa ett formulär som går snabbt att fylla i men som samtidigt inkluderar den mest relevanta informationen för att personal ska kunna bemöta den intresserade på rätt sätt. Jag valde även att infoga en separat knapp som leder till formuläret på startsidan tillsammans med en kort presentation av vad klubben har att erbjuda. Tanken med detta var att den som börjar läsa innehållet direkt ska hänvisas till formuläret.

5.1 Tester

Följande tester ska utföras på webbplatsen:

- Validering i W3C för både HTML och CSS.
- Funktionalitet i Chrome, Edge och Firefox.
- Tillgänglighetsanpassning, kontroll av navigering med tangenter, kontrast, alt-texter samt textstorlekar.
- Javascript ska fungera som planerat.
- Webbplatsen laddas inom tre sekunder.

6 Konstruktion

Konstruktionen av webbplatsen inleddes genom att skapa undersidornas struktur med hjälp av HTML-element. Eftersom alla undersidor har layouter som kräver att flera element placeras sida vid sida och lodrätt i förhållande till varandra ansåg jag det lämpligt att försöka planera sektioner och div-element för flexbox och css-grid redan från start. [6]

Webbplatsens index inleds med en hero tillsammans med en överliggande div innehållande klubbens slogan. På liten skärm är hero placerad under headern med hjälp av position absolute och ett negativt z-index för att minimera avståndet till sidans huvudinnehåll. I herons övre högra hörn ligger den utfällbara hamburgermenyn. Denna är satt till display none i koden för liten skärm men sätts till display block genom en eventlyssnare på klick i javascript. I eventlyssnaren finns en if-sats som säger att om display redan har värdet block ska det återgå till none vilket i praktiken gör att menyn stängs på klick två. Se figur 1.

```
const menuButton = document.querySelector("#menu");
const nav = document.querySelector("#nav");

function showNav() {
  if (nav.style.display === "block") {
    nav.style.display = "none";
  } else {
    nav.style.display = "block";
  }
}

menuButton.addEventListener("click", showNav);
```

Figur 1

Sidans FAQ styleades med den mörkare lila färgen som bakgrund med vit text över för god kontrast. För att lättare se vilken fråga man hovrar över och vilken som är aktiv efter klick skapades klasserna question:hover samt active som ger den aktuella frågan en ljusare lila bakgrund. Klassen active läggs till och tas bort genom växlingsmetoden toggle efter inspiration från w3school och deras teknik för sk. Accordions. [7]

Undersidan som visar ridskolans hästar är strukturerad med hjälp av css-grid. Valet föll på grid för att kunna sätta ett bestämt antal kolumner men samtidigt låta antalet rader vara auto för att layouten skulle vara densamma oavsett antal hästar. Kolumnerna är fyra på stor skärm för att minska skrollandet samt två på liten skärm för att behålla bilderna i relativt stor storlek. På denna sida finns även den tidigare nämnda funktionen för filtrering av hästarna.

Funktionen som styr filtreringen är skapad som en if-sats där varje value triggar filtrering av arrayen allHorseCards och endast returnerar de hästar som

uppfyller specifika kriterier. Det returnerade värdet sparas sedan i variabeln filteredArray som skriver över den tidigare arrayen. Vid filtreringen för ponnyer kontra storhästar användes den inbyggda funktionen ”includes” satt till att filtrera om ordet ”ponny” är inkluderat eller exkluderat i egenskapen breed, se figur 2. Den filtrerade arrayen skickas sedan med som ett argument in i funktionen som skriver ut hästarna till DOM på sidan.

```
function filterHorses() {
  // ny variabel för filtrerad array
  let filteredArray = allHorseCards;

  // filtrerar array beroende på value i selektorbox
  if (filterSelect.value === "gelding") {
    filteredArray = allHorseCards.filter((horse) => {
      return horse.gender === "Valack";
    });
  }

  if (filterSelect.value === "mare") {
    filteredArray = allHorseCards.filter((horse) => {
      return horse.gender === "Sto";
    });
  }

  if (filterSelect.value === "pony") {
    filteredArray = allHorseCards.filter((horse) => {
      return horse.breed.includes("ponny");
    });
  }

  if (filterSelect.value === "horse") {
    filteredArray = allHorseCards.filter((horse) => {
      return !horse.breed.includes("ponny");
    });
  }

  // skickar filtrerad array till display horsecard
  displayHorseCard(filteredArray);
}
```

Figur 2

På undersidan stallinfo finns den interaktiva karta som nämnts i avsnitt 5. Denna karta är en SVG-fil med en path för varje hage. SVG-filen skapades i vectorpea och koden kopierades sedan in till undersidans HTML-dokument. Filformatet SVG är lämpligt för detta ändamål då varje path skalas med bilden om fönstrets storlek förändras när attributet viewbox är tillämpat. [8] Varje hages id hämtades med queryselector och fick variablerna pasture1-6. Variablerna lagrades i en array för att kunna sätta en eventlyssnare för klick på varje hage genom en forEach-loop. Vid klick anropas funktionen pastureOpen med den klickade hagens variabel som argument. Det id som finns i variabeln innehåller samma siffra som står på hagen på kartan och jämförs därför med det nummer varje häst har på egenskapen ”pasture” i json-filen. Endast namnen på hästarna med matchande nummer skrivs ut till DOM. I funktionen finns även ytterligare en loop som summerar antal kilo dessa hästar ska äta på morgonen för att sedan skriva ut den totala hömängden till DOM. Se figur 3.

```
allHorseCards.forEach((horse) => {
  if (data.id.includes(horse.pasture)) {
    const nameP = document.createElement("p");
    nameP.textContent = horse.name;
    horsesDiv.appendChild(nameP);
  }
});
let sum = 0;
allHorseCards.forEach((horse) => {
  if (data.id.includes(horse.pasture)) {
    sum = sum + horse.hay.morning;
  }
});
```

Figur 3

Det är även på denna undersida som Open-meteos öppna API används för att skriva ut den dagliga väderprognosen. APIet hämtas med fetch och skickar sedan datan till funktionen displayWeather och displayBlankets. Genom den förstnämnda anropas sedan funktionen medianTemp som kalkykerar den genomsnittliga temperaturen för dagen. Denna returnerar ett värde genom den inbyggda funktionen toFixed som begränsar antalet decimaler och satt detta till endast en decimal. [9] Vädret placerades i en div som har vit bakgrund för att skapa kontrast mot de beigea sektionerna. Funktionen weatherType beräknar vilken ikon som ska användas för dagens väder baserat på den weather code som hämtas från open-meteo, se figur 4.

Att vädret visas på sidan är dock främst tillagt för sidans estetiska intryck. Den verkligt användbara funktionen som använder APIets data är funktionen displayBlankets som skriver ut vilka hästar som ska ha täcke på sig den givna dagen. Funktionen använder den returnerade genomsnittliga temperaturen samt weather code i en if-sats där varje scenario kör olika filtreringar av arrayen allHorseCards. De hästar som filtreras ut lagras i variabeln blanketsYes och skrivs sedan ut till DOM på sidan. Om antalet lagrade hästar är 0 skrivs ”ingen” ut och om antalet är 16 skrivs ”alla” ut.

Undersidans sista funktion är listan med uppgifter som ska göras under en dag. Denna del utgörs av knappar som vid klick flyttas ned till ”utfört” och blir grönmarkerade. Varje knapp har en eventlyssnare för klick där den klickade knappen skickas till en ny array som lagrar de utförda uppgifterna. Arrayerna sparas i localstorage och läses in på nytt vid omladdning för att det ska gå att fortsätta där man var sedan tidigare. Knappen ”börja om” återställer listan och gör den återanvändbar.

Webbplatsens sista funktion ligger på sidan kontakt och består av ett kontaktformulär. Detta valideras med hjälp av javascript som visar felmeddelanden om fält är tomma, om inmatad ålder är för låg eller om telefonnumret är för kort. För att göra detta har value från varje input hämtats in i en if-sats tillsammans med den inbyggda funktionen trim som tar bort blanksteg i början och slutet av inmatningen. [10] De felmeddelanden som

genereras pushas till arrayen errors och skrivs sedan ut på sidan som en lista med röd text. Om arrayen är tom efter alla if-satser testats returnerar valideringen true. Formuläret skickas då i väg och besökaren ser i stället texten ”tack för ditt meddelande”.

Webbplatsen innehåller en media query som är satt till min-width 768px. Just denna gräns valdes då surfplattor i landskapsorientering som minst brukar vara 768px och uppåt. [9] Originalutförandet av webbplatsen är skapad enligt mobile first och denna media query är därmed tänkt att användas för alla skärmar större än en telefon. Viljan att addera ännu en media query specifikt för surfplattorna har funnits, men med tanke på tidsåtgång till övriga delar i projektet har detta behövt prioriteras ned.

7 Resultat

Resultatet är en webbplats som innehåller fyra undersidor inklusive index. På stor skärm visas navigeringsmenyn mellan dessa som fyra klickbara länkar och på liten skärm kan dessa visas eller döljas genom en sk. Hamburgermeny.

På index finns en knapp för att komma direkt till ridklubbens kontaktformulär. Kontaktformuläret innehåller sju inputfält och valideras med hjälp av javascript.

Undersidan för ridklubbens hästar visar sexton individuella hästar som även kan filtreras efter kön eller storlek. Vid klick på en häst visas en dialogruta med all information som finns lagrad i json-filen för hästen. Informationen som hämtas gällande foder och täcken visas i varsin tabell. Dialogrutan öppnas och stängs med hjälp av javascript.

På undersidan stallinfo finns tre huvudsakliga sektioner med olika funktion. Den centrerade diven på sidan visar en enkel karta över stallets hagar. Vid klick på en utav hagarna visas en dialogruta med vilka hästar som går i hagen samt hur mycket hö dessa hästar ska äta tillsammans på morgonen. Dialogen öppnas och stängs med javascript.

På den högra sidan om kartan finns en sektion som visar dagens generella väder och temperatur. Utifrån detta visas en lista med namnen på de hästar som behöver ha täcke på sig för dagen. På kartans vänstra sida finns en sektion med en klickbar att göra-lista. De klickade uppgifterna blir gröna och förflyttas ned till en underliggande lista med rubriken ”utförda”. Listan sparas i localstorage till dess att användaren klickar på knappen ”börja om”.

7.1 Testresultat

- CSS validerar utan fel. HTML validerar utan fel men med flera likadana tips. Dessa beror på formateringstillägget prettier i Visual Studio Code som lägger till snedstreck innan sluttagg på öppna element. Detta valde jag att bortse från.
- Webbplatsen fungerar i både chrome, edge och firefox.
- Krav på kontrast enligt WCAG 2.0 level AA uppfylls. Webbplatsen går att navigera med tab. Text förstoras vid zoom och bilder innehåller alt-attribut.
- Javascript fungerar enligt plan för de funktioner som tidigare räknats upp.
- Webbplatsen laddas inom tre sekunder.

8 Slutsatser

Slutsatsen jag drar av detta arbete är att de huvudsakliga funktionerna på webbplatsen faktiskt fungerar som tänkt. Jag stötte på en del problem gällande kartan som i planeringsfasen var tänkt att skapas som en html image map, men det visade sig snabbt att det inte rimmar väl med en responsiv webbdesign då de klickbara ytorna inte skalar med bilden. Lösningen fick bli att skapa en SVG-fil för att agera karta i stället för den tänkta bilden. Med tanke på mina begränsade kunskaper när det kommer till filformatet SVG så blev denna väldigt enkel och jag insåg att en dålig lagerstruktur återspeglar sig direkt i bildens kod.

Jag har haft mycket huvudbry kring textstorlekar i detta projekt och har fått inse att jag behöver mycket mer övning på att förstå vilken enhet som lämpar sig var och när. I projektets början användes nästan uteslutande vw då jag nyligen upptäckt att det är en responsiv enhet, men den fungerar dåligt om man jobbar mobile-first och sedan skalar upp fönstret till desktop då allt känns gigantiskt. Jag lärde mig då att använda clamp där man kan sätta ett minsta- och största storleksvärde och sedan ett dynamiskt värde mellan dessa. [10] Jag upplevde att denna metod löste problemet genom att kunna använda enheten rem för att begränsa texten och ett vw-värde i mitten som gjorde texten responsiv.

Javascripten fungerar enligt mina mått mätt bra och inkluderar både ett API och en funktion som använder sig av localstorage vilket var något jag var beslutad att få in redan från början. I planeringsfasen gjorde jag dock en tankevurpa då stora delar av de planerade funktionerna byggde på att localstorage sparas för alla som besöker webbplatsen. När jag kom på att datan troligen bara sparas för den enskilda användaren fick jag tänka om men trots detta är jag nöjd med resultatet. De idéer som fick kasseras var någon form av inloggningsmöjlighet för admin för att kunna uppdatera information samt via denna sida en möjlighet att skriva ut en lista på vilken elev som skulle rida vilken häst inför varje dag. Detta hade dock krävt någon form av lagring i databaser vilket just nu är över min kompetens. När mockupen skapades ersattes platsen denna funktion legat på av en knapp som skulle leda till en dagbok för skötare där man själv skulle kunna föra dagbok om sin häst. Tiden blev dock knapp och jag valde därmed att bortprioritera även denna funktion.

Styling och CSS blev som jag tänkte mig och jag upplever designen som relativt stilren utan att vara tråkig. Min vilja att ge bilderna på hemsidan mest fokus och utrymme har enligt mig givit det resultat jag förväntat mig.

Detta projekt hade kunnat fortsätta i all evighet då jag har många tankar och idéer på funktioner på just detta tema. För mig blev det därför nästan viktigare att dra en gräns i tid för att inte ta mig vatten över huvudet och addera fler funktioner än vad jag kunde åstadkomma som amatör på den givna tiden.

Källförteckning

[1] Svenska ridsportsförbundet
https://ridsport.se/ridskola
Publicerad 2026. Hämtad 2026-10-29.

[2] W3Schools
https://www.w3schools.com/html/html5_api_whatis.asp
Hämtad 2026-11-01

[3] Kursmaterial ”Introduktion till webbutveckling med html, css och javascript” (DT224G)
Hämtad 2026-11-01

[4] Google AdSense Help
https://support.google.com/adsense/answer/7450973?hl=en
Publicerad 2026. Hämtad 2026-10-29.

[5] Shoot it Film & Foto
https://shootit.se/basta-bildformat-for-hemsidor/
Publicerad 2024-03-19. Hämtad 2026-10-29.

[6] W3Schools
https://www.w3schools.com/css/css3_flexbox.asp
Hämtad 2026-10-30

[7] W3Schools
https://www.w3schools.com/howto/howto_js_accordion.asp
Hämtad 2026-10-30

[8] Amelia Bellamy-Royds
https://css-tricks.com/scale-svg/
Publicerad 2015-01-06. Hämtad 2026-10-30.

[9] W3Schools
https://www.w3schools.com/jsref/jsref_tofixed.asp
Hämtad 2026-10-30

[10] Mozilla Developer Network
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/trim
Hämtad 2026-10-31

[11] W3Schools
https://www.w3schools.com/css/css_rwd_mediaqueries.asp
Hämtad 2026-10-31

[12] W3Schools
https://www.w3schools.com/cssref/func_clamp.php
Hämtad 2026-10-31

[13] W3Schools
https://www.w3schools.com/whatis/whatis_json.asp
Hämtad 2026-11-01